

DSM500 Final Project Report

Synthetic Markets, Real Decisions: Evaluating the Downstream Utility of Generative Models for Trading-Strategy Validation

Author: Guillermo Enrique Olmos Ranalli (200153474)

Module: DSM500 - Data Science MSc Final Project

Date: October 2026

Contents

- [Abstract](#)
- [1. Introduction](#)
 - [1.1 Motivation: The Backtesting Problem](#)
 - [1.2 A Natural Solution: Synthetic Market Data](#)
 - [1.3 The Critical Gap: Statistical Realism \$\neq\$ Task Relevance](#)
 - [1.4 Aim and Objectives](#)
 - [1.5 Research Questions](#)
 - [1.6 Stakeholders](#)
 - [1.7 Preview of Findings](#)
- [2. Literature Review](#)
 - [2.1 Backtest Overfitting and Data Snooping](#)
 - [2.2 Stylized Facts as the Yardstick for Realism](#)
 - [2.3 Classical Generators: Resampling and Volatility Models](#)
 - [2.4 Deep Market Generators](#)
 - [2.5 How Synthetic Financial Data Are Evaluated](#)
 - [2.6 Gap and Positioning](#)
- [3. Methods and Experimental Design](#)
 - [3.1 Data](#)
 - [3.2 Windowing Strategy](#)
 - [3.3 Generators Evaluated](#)
 - [3.4 Evaluation Metrics](#)
 - [3.5 Trading Rules Universe](#)
 - [3.6 Testing and Verification](#)
 - [3.7 Reproducibility](#)
 - [3.8 Implementation](#)
- [4. Results](#)
 - [4.1 RQ1: Stylized Facts Fidelity](#)
 - [4.2 RQ2: Downstream Task-Based Utility \(Primary Result\)](#)
 - [4.3 RQ2 Secondary Metrics: False Discovery, Optimism and Selection Regret](#)
 - [4.4 Robustness and Paired Comparison of Generators](#)
 - [4.5 Sensitivity by Rule Family](#)
 - [4.6 Summary of Results](#)
- [5. Discussion](#)
 - [5.1 Why Does Matching History Not Preserve Later Rankings?](#)
 - [5.2 Why the Deep Generators Do Not Demonstrate Utility](#)
 - [5.3 Relation to Prior Work](#)
 - [5.4 Limitations](#)
 - [5.5 Changes from the Proposal](#)
 - [5.6 Reflection](#)
- [6. Conclusions](#)

- 6.1 Answering the Research Questions
- 6.2 Contribution
- 6.3 Practical Implications
- 6.4 Future Work
- 7. Ethical Considerations
- 8. References
- 9. Appendices
 - A. Data Summary
 - B. GAN Training
 - C. Rule Universe
 - D. Code Availability
 - E. Code Listings

Abstract

This project evaluates generative models of financial returns by their downstream usefulness for trading-strategy validation. Daily USD/EUR returns are split into training data from 2018–2021 and held-out data from 2022–2024. Four generators provide 1,000 synthetic paths each: a window bootstrap, a variance-targeted GARCH(1,1), an MLP GAN and an LSTM GAN. A universe of 500 uniquely named rule specifications is evaluated using daily position-based Sharpe ratios. The synthetic rankings are compared with rankings on 526 held-out windows and with a chronological historical-validation baseline.

Main finding. The corrected experiment finds no detectable positive ranking signal from GARCH (Spearman $\rho = 0.041$, 95% CI -0.048 to 0.130), LSTM GAN (0.031 , -0.058 to 0.120) or MLP GAN (-0.020 , -0.109 to 0.069). Window bootstrap (-0.306) and historical validation (-0.286) are negatively associated with later rankings. A paired comparison on the 484 rules common to all methods places GARCH and both GANs above the negatively ranked historical methods, but cannot separate GARCH from either GAN. This relative improvement does not establish useful positive prediction. Closely related rules make the resampling intervals optimistic.

Fidelity and decision quality. Window resampling closely matches training moments but produces a negative downstream correlation. GARCH broadly matches the return scale but under-represents tails; the saved GANs have distorted dependence or scale. No generator demonstrates reliable strategy selection in this case study.

Contribution and limits. The contribution is a reproducible task-based evaluation and an informative negative result. A final audit corrected trade-return accounting, rule identifiers, single-asset selection, a GARCH recursion defect and a simulated baseline; the numbers reported here were regenerated after those corrections. The GAN checkpoints were retained rather than retrained and remain limited by pooled, zero-filled training inputs. The study mainly concerns moving-average rules, one currency quote and one test period; transaction costs and synthetic ground-truth controls remain unimplemented. The results provide no investment recommendation.

1. Introduction

1.1 Motivation: The Backtesting Problem

Backtesting—evaluating trading strategies on historical data before deployment—is the central tool in quantitative finance. Yet it is notoriously unreliable. Because strategies are tuned on a single realised price path, strong in-sample performance frequently fails to persist out-of-sample, a manifestation of overfitting and data-snooping bias documented extensively in the literature (White, 2000; Bailey et al., 2017; Harvey, Liu and Zhu, 2016).

The consequences are severe:

- Capital is deployed to strategies whose apparent edge is an artifact of noise
- The published record of "profitable" strategies is inflated by selection bias from configurations silently tested and discarded
- Harvey, Liu and Zhu (2016) argue that, once the scale of searching is accounted for, many published findings about expected returns are likely to be false discoveries

1.2 A Natural Solution: Synthetic Market Data

If we could generate many plausible alternative price paths—each realistic in its statistical properties but different from history—we could validate whether strategy performance is robust across them. Recent advances in deep generative models (GANs, diffusion models) make this possible. A growing body of work shows such "market generators" can reproduce salient stylized facts of real returns: heavy tails, volatility clustering, near-zero autocorrelation (Wiese et al., 2020; Takahashi, Chen and Tanaka-Ishii, 2019).

1.3 The Critical Gap: Statistical Realism $\neq$ Task Relevance

However, prior work evaluates generators mainly on statistical fidelity—how closely synthetic data matches moment distributions, autocorrelations, and other stylized facts. This evaluation approach has a blind spot: **a generator can match every statistical property and still be useless (or actively misleading) for a downstream decision, because realism in aggregate statistics does not guarantee that the specific temporal dependence a strategy exploits is preserved.**

Assefa et al. (2019) note that there is still no agreed way to evaluate synthetic financial data, and that statistical-distance measures cannot be relied on as evidence of realism. This project takes a complementary, task-based route: we compare generators not just on how realistic they are, but on whether strategies validated on synthetic data actually predict real out-of-sample performance—the decision that matters to practitioners.

1.4 Aim and Objectives

Aim. To develop and evaluate a task-based framework that measures generative market models by the quality of the strategy-validation decisions they support, and to establish whether deep generators offer more validation utility than classical resampling and volatility-modelling baselines.

Objectives.

1. Implement deep generative models of daily return windows (an MLP GAN and an LSTM GAN) alongside two classical baselines (GARCH(1,1) and a window bootstrap).
2. Evaluate each generator's statistical fidelity with stylized-fact diagnostics (RQ1).
3. Build a task-based protocol that backtests a universe of trading rules on each data source and measures how well the resulting ranking predicts held-out real performance, with complementary decision metrics (RQ2).
4. As an extension, examine how this utility varies across assets and market regimes (RQ3).
5. Deliver the pipeline in a reproducible, tested form.

Section 5.5 records where the delivered study departs from these objectives.

1.5 Research Questions

The project is organised around three comparative research questions:

- **RQ1:** How faithfully do deep generative models reproduce the stylized facts of real financial time series, relative to GARCH and bootstrap baselines?
 - *Rationale:* Establishes that statistical fidelity is achievable; baseline for comparison to RQ2.
- **RQ2 (Primary):** To what extent does validating trading strategies on generated data improve the accuracy of out-of-sample performance estimates and reduce false discovery, compared with single-history walk-forward validation and classical resampling?
 - *Rationale:* Measures downstream decision quality—the core innovation. Does synthetic validation actually help?
- **RQ3 (Exploratory):** How does the downstream utility of each validation method vary across asset classes and market regimes?
 - *Rationale:* Explores generalisation; deferred to future work if time-constrained.

1.6 Stakeholders

Several groups stand to benefit from a reliable, task-based way to validate trading strategies.

Quantitative researchers and strategy developers gain a check on whether an apparently profitable backtest is likely to survive out of sample. **Risk and investment committees**, who decide whether to allocate capital, gain an interpretable measure of how far a validation

method can be trusted and how much it inflates expectations. **Practitioners weighing tooling costs** gain evidence on whether expensive deep generators are worth their cost or whether cheap baselines suffice. The **academic and open-source community** gains a reusable framework that judges market generators by decision usefulness rather than statistical realism alone. The project makes no investment recommendation; its value lies in better-informed validation, not in trading signals.

1.7 Preview of Findings

The corrected experiment compares four synthetic-data generators and a chronological historical-validation baseline. Every rule is evaluated on the same held-out USD/EUR period, with pairwise statistics restricted to observations available for the relevant methods. There are 500 uniquely named specifications, of which 488 have defined real Sharpe ratios; 484 are available for the comparison involving every method.

RQ1. The window bootstrap reproduces the training moments closely: mean return autocorrelation over lags 1–20 is -0.0118 for real windows and -0.0116 for resampled windows, while excess kurtosis is 2.48 and 2.47. GARCH's mean daily standard deviation is close to the observed one, but its tails are much thinner. Both saved GANs differ substantially from the data in scale or dependence. A stationary bootstrap is also measured as a fidelity comparator, although the downstream experiment uses whole-window resampling.

RQ2. GARCH's correlation is 0.041, the LSTM GAN's 0.031 and the MLP GAN's -0.020 ; none is distinguishable from zero by the reported marginal intervals. Window bootstrap and historical validation give negative correlations, -0.306 and -0.286 . The paired comparison distinguishes those historical methods from GARCH and the GANs but does not identify a positive-signal winner among the three latter generators.

Methodological point. A generator can be preferable to a poor comparator without providing a useful positive ranking. Fidelity, absolute predictive usefulness and relative performance must therefore be reported separately. The corrected results replace an earlier apparent GARCH advantage that depended on defective calculations, illustrating why a plausible result is insufficient evidence of a valid experiment.

2. Literature Review

This review brings together four strands of work: the statistics of backtest overfitting, the stylized facts used to judge market realism, the generators that produce synthetic markets, and the way those generators are evaluated. The aim is to show where they meet and where they leave a gap.

2.1 Backtest Overfitting and Data Snooping

That backtests overstate future performance is well established. White (2000) frames the problem as data snooping: when many rules are tried on the same data, the best one looks good partly by chance, and his Reality Check uses a bootstrap to test whether the best rule beats a benchmark once the search is accounted for. Harvey, Liu and Zhu (2016) make the same argument for the cross-section of expected returns. Given the hundreds of factors already tested, they argue, the conventional significance threshold is far too lenient and a much higher hurdle is needed. Bailey and López de Prado (2014) translate the idea into a performance measure, the Deflated Sharpe Ratio, which discounts an observed Sharpe ratio for the number of trials and for non-normal returns. Bailey et al. (2017) estimate the Probability of Backtest Overfitting, the chance that the configuration chosen in-sample underperforms out of sample, using combinatorially symmetric cross-validation. López de Prado (2018) restructures validation itself, proposing purged and combinatorial cross-validation to limit leakage between training and test folds.

These methods differ in what they deliver: a significance test (White; Harvey, Liu and Zhu), an adjusted performance figure (the Deflated Sharpe Ratio), a probability (PBO) or a validation design (purged cross-validation). They share one feature: all of them work with the single observed history. They correct or discount a selection after it has been made, but cannot say how a rule would have behaved under different but plausible histories. Synthetic data offers that complementary route, by enlarging the set of histories on which rules are validated. It is only useful if the synthetic histories preserve whatever makes rules succeed or fail, which is the question this project tests.

2.2 Stylized Facts as the Yardstick for Realism

Cont (2001) catalogues the statistical regularities of asset returns: heavy tails, near-zero linear autocorrelation, volatility clustering (slowly decaying autocorrelation of absolute or squared returns), the leverage effect, gain/loss asymmetry and aggregational Gaussianity. These properties have become the standard yardstick for synthetic market data. A generator is considered realistic if its returns reproduce them.

For strategy validation the yardstick has two weaknesses. First, stylized facts are low-order summaries: a generator can match the distribution of returns and their autocorrelations while still misplacing the trend persistence or the timing of volatility bursts on which a trading rule's profit depends. Second, they are hard to turn into a single score. Assefa et al. (2019)

observe that stylized facts are often assessed qualitatively and had not yet been converted into a convincing single real-valued metric. Realism is therefore assessed piecemeal, and its link to downstream decisions is rarely tested.

2.3 Classical Generators: Resampling and Volatility Models

Two classical families are natural baselines. The stationary bootstrap of Politis and Romano (1994) resamples blocks of random length, which preserves short-range dependence while keeping the resampled series stationary. Resampling methods reproduce the empirical distribution exactly and need no model, but they cannot produce events outside the observed sample, and dependence longer than the blocks is lost. The window bootstrap used in this project is the extreme case: it resamples whole 256-day windows, so each synthetic path is a piece of real history.

The GARCH model of Bollerslev (1986) instead models the conditional variance recursively. A large shock raises tomorrow's variance, which produces volatility clustering and, even with Gaussian innovations, fat-tailed unconditional returns. The symmetric GARCH(1,1) does not capture the leverage effect, and with Gaussian innovations it usually produces less excess kurtosis than observed. Assefa et al. (2019) summarise the trade-off: autoregressive and GARCH-type models are easy to fit and interpret but rest on strong assumptions and miss many features of real data. Both families are cheap, transparent and well understood, which makes them the natural benchmark any more complex generator should be required to beat.

2.4 Deep Market Generators

Generative adversarial networks (Goodfellow et al., 2014) train a generator to fool a discriminator, and several studies adapt them to financial time series. Takahashi, Chen and Tanaka-Ishii (2019) show that a GAN can reproduce several stylized facts of financial returns. Wiese et al. (2020) introduce Quant GANs, which use temporal convolutional networks to capture long-range dependence such as volatility clustering. They evaluate the generated series with distributional distances and stylized-fact comparisons. Yoon, Jarrett and van der Schaar (2019) propose TimeGAN for general time series, adding a supervised step-ahead loss so that the model learns temporal dynamics rather than only the distribution of each time step. Notably, they assess generated data not only for fidelity but for usefulness, through a predictive score: a sequence model is trained on synthetic data and tested on real data. Denoising diffusion models (Ho, Jain and Abbeel, 2020) offer an alternative to adversarial training and have since been adapted to time series. Diffusion-TS (Yuan and Qiao, 2024) generates general multivariate series with an encoder-decoder transformer and an interpretable trend and seasonal decomposition. FTS-Diffusion (Huang, Chen and Qiao, 2024) targets financial series specifically. It extracts recurring patterns that vary in duration and magnitude, synthesises them with a diffusion network and models their succession; augmenting real data with its samples reduced stock-prediction error by up to 17.9% in the authors' experiments. Diffusion models were not implemented in this project (Section 5.5).

Two points stand out. First, progress is measured mainly by resemblance to real data. Where usefulness is tested, the downstream task is prediction: a forecasting model trained on, or augmented with, synthetic data (Yoon, Jarrett and van der Schaar, 2019; Huang, Chen and Qiao, 2024). None of these studies asks whether synthetic data lead to better selection of trading strategies. Second, deep generators are data-hungry and can be unstable to train, while daily financial series offer only a few thousand observations per asset. This tension matters for the results in Section 4.

2.5 How Synthetic Financial Data Are Evaluated

Assefa et al. (2019) survey synthetic data in finance, with a strong emphasis on privacy, and identify measuring the similarity between real and generated data as one of three open problems. They note that the field has no common benchmarks, datasets or metrics. Drawing on work in image generation, they argue that statistical-distance measures cannot be relied on as metrics of realism. They also observe that backtest-style evaluations, such as comparing a generator's Value-at-Risk with the value later observed, suffer from a dearth of data because only one history is available. Among the motivations for synthetic data they list the lack of historical data and the usefulness of counterfactual data for testing strategies.

Synthetic data have also been used directly for backtesting. López de Prado (2018, ch. 13) fits a discrete Ornstein-Uhlenbeck process to historical prices, generates many synthetic paths and uses them to choose profit-taking and stop-loss thresholds, arguing that this greatly reduces backtest overfitting compared with a brute-force search on the single history. There the generator is one fixed parametric model and the task is calibration; competing generators are not compared.

Koshiyama, Firoozye and Treleaven (2021) come closest to a task-based evaluation of deep generators for trading. They use conditional GANs to calibrate the hyperparameters of trading strategies and to combine strategies into ensembles, testing on 579 assets. They report that the approach can outperform when traditional techniques fail to generate alpha. Their evaluation is task-based, but its question is profitability: does tuning or combining with GAN samples make money? It does not ask whether rankings on synthetic data predict rankings on unseen real data, and it does not compare the GAN against cheap resampling or volatility baselines on that question.

2.6 Gap and Positioning

Taken together, the overfitting literature corrects selection on a single history. The generator literature measures mainly resemblance, and its task-based evaluations concern prediction. The evaluation literature records that no agreed metric exists. Synthetic backtesting has been used to calibrate rules under one parametric model (López de Prado, 2018) and to tune and combine strategies with GANs (Koshiyama, Firoozye and Treleaven, 2021). What is missing is a direct comparison of competing generators, classical and deep, on whether their synthetic backtests rank trading rules the way unseen real data later does.

This project fills that gap at pilot scale: a single asset (USD/EUR), a mainly moving-average rule universe and one test period. Consistent with the efficient-market view that price changes are hard to predict (Fama, 1970), it studies the quality of validation, not the discovery of profitable rules.

3. Methods and Experimental Design

Figure 3.1 summarises the evaluation workflow; the subsections below describe each stage.

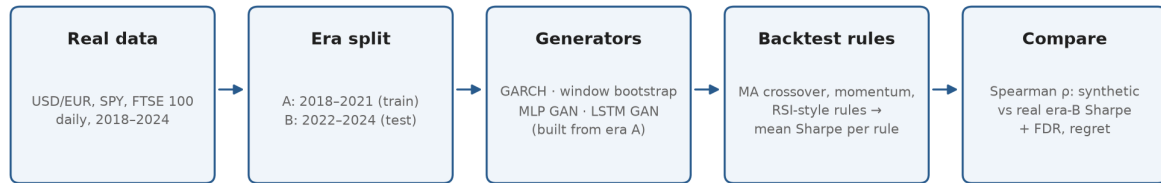


Figure 3.1. Workflow. Generators are built from era-A data; every rule is backtested on synthetic paths and on real era-B data, and the two sets of Sharpe ratios are compared.

3.1 Data

Sources and assets:

- **USD/EUR**: exchange rate ($\text{EUR}=\text{X}$), downloaded from Yahoo Finance (Yahoo Finance, 2026) with the open-source yfinance library (Aroussi, n.d.)
- **SPY**: S&P 500 exchange-traded fund
- **FTSE 100**: UK equity index (^FTSE)
- All daily closing prices, 2018–2024 (7 years), from the same source

Era split (chronological, before windowing):

- **Era-A (2018–2021, training)**: about 1,010–1,040 daily returns per asset (788 USD/EUR windows)
- **Era-B (2022–2024, held-out test)**: about 750–780 daily returns per asset (526 USD/EUR windows)
- Split prevents look-ahead leakage; era-B is entirely unseen during training

Window and return counts for every asset and era are given in Appendix A (Table A.1).

Choice of instruments. The proposal planned hourly cryptocurrency data as the primary series, with a daily cross-asset arm. After the cryptocurrency download failed (Section 5.5), the daily arm became the study. USD/EUR, SPY and the FTSE 100 are liquid daily series without survivorship bias, covering three distinct markets: a major currency pair, a US equity fund and a UK equity index. Because of time constraints, only USD/EUR was evaluated; SPY and the FTSE 100 were prepared for the cross-asset extension (RQ3).

Quote convention. The USD/EUR series is Yahoo Finance's $\text{EUR}=\text{X}$ quote, which gives euros per US dollar (Yahoo labels it USD/EUR). Its log returns are therefore the negative of the conventional EUR/USD rate, and the long-only rules in this study are long the dollar against the euro. Throughout the report, "USD/EUR" refers to this series as downloaded.

Choice of eras. The chronological split places 2018–2021 in era A and 2022–2024 in era B, so that the held-out period is a different market regime rather than a further sample of the same one. Table 3.1 and Figure 3.2 show that this holds for USD/EUR. Era B has higher volatility (7.9% against 6.4% annualised) and stronger volatility clustering (lag-1 autocorrelation of squared returns 0.10 against 0.03), with its most volatile stretch in the first year of era B. For the two equity series the ordering is reversed, because era A contains the 2020 market shock. A four-year training era gives about 1,000 daily returns per asset, enough for 256-day windows, while leaving three years for testing.

Table 3.1. Daily log-return statistics by asset and era. Series rebuilt from the windows (`pipeline/14_data_description.py` , `outputs/data_description.json`).

Asset	Era	Returns	Annualised volatility	Skewness	Excess kurtosis	Max drawdown	Autocorr. of squared returns, lag 1
USD/EUR	A (2018–21)	1,043	6.4%	0.37	2.9	-13.6%	0.03
USD/EUR	B (2022–24)	781	7.9%	0.01	1.2	-14.6%	0.10
SPY	A (2018–21)	1,007	20.9%	-1.05	15.9	-33.7%	0.49
SPY	B (2022–24)	752	17.5%	-0.23	1.8	-24.5%	0.10
FTSE 100	A (2018–21)	1,011	18.3%	-1.22	16.4	-36.6%	0.15
FTSE 100	B (2022–24)	754	12.8%	-0.45	3.5	-11.0%	0.23

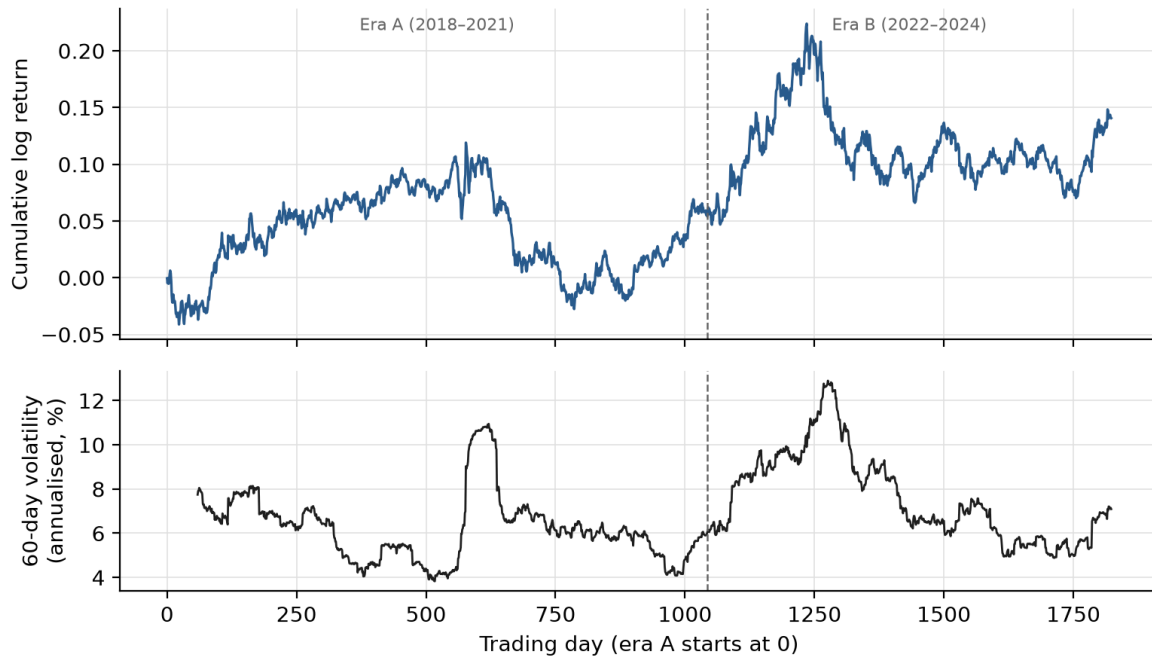


Figure 3.2. USD/EUR (EUR=X) cumulative log return and 60-day annualised volatility; the dashed line marks the era split. The windows carry no dates, so the axis counts trading days.

Table 3.1 describes each reconstructed daily sequence, whereas Table 4.1 averages statistics over overlapping 256-day USD/EUR windows. These weighting and sample-length differences explain why their moment estimates differ. The final fidelity comparison no longer mixes currencies and equity indices into one reference distribution.

Evaluation focus: Evaluation focused on USD/EUR (columns 0–255 of 768-dim data); SPY and FTSE prepared but not included in RQ2 analysis due to time constraints. This represents a focused case study on a single asset. The three instruments were chosen as liquid daily series without survivorship bias, covering a currency pair, a US equity fund and a UK equity index. The stored data hold one asset per row in a 768-column layout (3 assets × 256 steps); columns belonging to the other two assets are empty. They were zero-filled for the saved GAN training, but the corrected bootstrap, GARCH and real-data evaluations select only USD/EUR rows and columns (Section 5.4).

Returns and preprocessing:

- Log returns: $r_t = \ln(P_t / P_{t-1})$
- 256-step windows with stride 1 (maximum overlap)
- Underlying return sequences, era-A: 1,043 (USD/EUR), 1,007 (SPY), 1,011 (FTSE 100)
- Underlying return sequences, era-B: 781 (USD/EUR), 752 (SPY), 754 (FTSE 100)

3.2 Windowing Strategy

Why 256 steps?

- Volatility clustering decays slowly, roughly as a power law in the lag (Cont, 2001), so long windows are needed to contain several volatility episodes
- 256 days $\approx$ 1 calendar year; captures multiple volatility cycles
- Aligns with technical analysis conventions (200–260-day moving averages)

Stride: 1 day (maximum overlap)

- Maximises sample count for generative model training
- Standard in deep learning (sliding windows in CNNs, LSTM datasets)

Leakage prevention:

- Windows created *after* era split
- No window contains data from both eras
- Boundary-straddling windows discarded

3.3 Generators Evaluated

Baseline 1: Window-Level Resampling (Nonparametric)

- Resamples entire 256-step windows with replacement
- Generates synthetic data by randomly selecting from empirical windows
- Nonparametric; preserves the empirical statistics of each window
- The RQ1 fidelity run used the same whole-window resampling (`02_baselines.py`)

Baseline 2: GARCH(1,1) (Bollerslev, 1986)

The conditional variance follows $h_t = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot h_{t-1}$; each innovation is then $\varepsilon_t = \sqrt{h_t} z_t$, with independent standard-normal z_t . The order matters: the variance must be updated before drawing the return shock using it. The corrected implementation includes a stochastic first innovation and is checked against an independent, hand-calculated recursion.

The parameters $\alpha = 0.05$ and $\beta = 0.94$ remain fixed. The mean and variance are estimated from the reconstructed 1,043 era-A USD/EUR returns, and ω is set to the sample variance multiplied by $1 - \alpha - \beta$. The resulting ω is 1.625×10^{-7} and the daily standard deviation is 0.00403. This is variance targeting, not maximum-likelihood estimation of all three parameters. Persistence is 0.99, corresponding to a variance-shock half-life of about 69 days. The parameter file is generated from that fit rather than entered by hand.

Additional fidelity comparator: stationary bootstrap. A stationary bootstrap draws geometrically distributed blocks with expected length 20 days from the reconstructed training sequence, wrapping at its boundary (Politis and Romano, 1994). It tests how resampling beyond intact windows changes fidelity. It is not included in the final RQ2 strategy-ranking comparison; the downstream resampling method remains the window bootstrap.

Primary: MLP GAN

- Generator: 4-layer MLP (100-dim noise → [256, 512, 256] → 768-dim tanh output)
- Discriminator: 4-layer MLP with dropout, LeakyReLU
- 50 epochs training; BCEWithLogitsLoss
- Per-epoch losses are stored in the model checkpoint and exported by `13_gan_training_history.py` (Appendix B)

Robustness check: LSTM GAN

- Generator: Bidirectional LSTM layers + linear projection (explicit temporal modelling)
- Discriminator: LSTM + classification head
- 50 epochs training
- Same pooled training data as the MLP GAN; both saved checkpoints are evaluated with the corrected single-asset backtester
- Neither checkpoint was retrained for the final rerun. Their first 256 output columns represent the currency block; limitations of learning from zero-filled, pooled inputs remain

3.4 Evaluation Metrics

RQ1: statistical fidelity. Each 256-day path supplies the mean autocorrelation of returns and squared returns over lags 1–20, excess kurtosis, skewness and daily standard deviation. Statistics are then averaged across windows, so every window has equal weight. The real reference consists only of era-A USD/EUR windows. The first Wasserstein distance is calculated using SciPy on the flattened real and generated window distributions; the real distribution's distance from itself is zero. This distributional measure gives repeated weight to observations shared across windows and is interpreted alongside, rather than as a replacement for, per-window dependence diagnostics.

The estimands are deliberately explicit. An autocorrelation averaged over twenty lags is not a lag-1 autocorrelation, and mean window kurtosis is not the kurtosis of a concatenated series. Values from these alternatives cannot be interchanged. The descriptive table uses each underlying return once, while the fidelity table gives equal weight to heavily overlapping windows. Matching a few averages also cannot establish realism for every temporal property that a rule might exploit.

RQ2: strategy-ranking utility. For each uniquely named specification, the same backtester generates a daily strategy-return stream on every real window and every synthetic path. It enters at the close of a signal day, so the first held return is the following day's return. It holds at most one position at a time. Profit and stop thresholds are checked on cumulative log returns since entry, and moving-average signals use the reconstructed cumulative log-price path. The per-window score is the mean daily position-weighted log return divided by its standard deviation, including uninvested days as zero returns. Scores are not annualised, and the risk-free rate and transaction costs are taken as zero.

A window with no position or zero return variation has an undefined Sharpe ratio and is omitted from that rule's average. This prevents division by zero but means the reported average is conditional on windows with a defined score. Methods and rules may therefore have different activity rates. The experiment does not estimate the Sharpe of one continuous seven-year portfolio, and averaging window Sharpes is not equivalent to calculating a Sharpe from concatenated returns. These design choices constrain the economic interpretation of every result.

Spearman correlation compares the mean validation score and mean real era-B score across rules. A positive coefficient would mean agreement, a coefficient near zero little evidence of ranking information, and a negative coefficient reversed ordering. The main table reports each method's available rules; paired comparisons use the common complete subset. Fisher-z intervals are approximate and the rule-resampling intervals assume independence between rules, although many specifications are close variants. They describe variation over this selected rule universe rather than uncertainty over all possible market histories.

Historical-validation comparator. The benchmark is computed by backtesting the rules on real era-A windows and comparing those rankings with real era-B rankings. It uses an actual earlier history and no synthetic perturbation of future outcomes. This is one chronological train/test comparison, not a sequence of rolling model refits; the file retains its earlier name, `07_walkforward_baseline.py`, but its displayed label is Historical validation. A negative result for it does not establish that every walk-forward design would fail.

Secondary decision metrics. A rule is endorsed when its validation score exceeds 0.1 in daily-Sharpe units. False discovery is the fraction of endorsed rules whose real score does not exceed that same threshold. It is undefined when no rules are endorsed; zero coverage cannot be described as perfect accuracy. This threshold is an illustrative screen, not a statistical significance test. Optimism is the validation score minus the real score, and selection regret is the difference between the best real score available in that method's valid subset and the real score of its validation-selected rule. Both are descriptive scores in daily-Sharpe units, not currency profits.

3.5 Trading Rules Universe

The generator produces 500 specifications with unique identifiers: 480 moving-average crossover rules, 16 momentum rules and four RSI-style rules. Stop-loss values are included in identifiers, preventing variants from being overwritten under a shared name. Distinct identifiers do not imply independent strategies or distinct realised trades: nearby parameter choices may produce identical positions on some paths. The 500 specifications therefore remain a tightly related candidate set, especially within the moving-average family.

Moving-average fast windows are 10, 15, 20, 25 and 30 days; slow windows are 50, 75, 100 and 150 days. Crossovers are calculated on cumulative log prices, with today's closing information determining tomorrow's exposure. Momentum thresholds range from 0.01 to 0.05 in daily log returns. The RSI-style rule is a percentile rule rather than a standard Wilder RSI

calculation: today's return is compared with an expanding past-return quantile, shifted by one day, with at least twenty observations. That distinction is retained in the family label and prevents attributing the results to a conventional RSI implementation.

Exits use fixed holds of 5–20 bars, or a combined maximum hold, profit target and stop loss. All positions are long-only in the dollar against the euro. The fixed generation order fills almost the entire set with moving-average rules; Bollinger-band rules are not reached. The four RSI specifications are early variants from the same sweep, and high momentum thresholds seldom trigger in daily currency data. There are 488 specifications with a defined held-out score, comprising 480 MA, four momentum and four RSI-style rules. Both GAN comparisons have defined scores for 484, excluding momentum. Family-specific conclusions beyond MA are therefore unsupported.

3.6 Testing and Verification

The pytest suite covers data integrity, split boundaries, baseline generators, identifier uniqueness, GAN interfaces and pipeline integration. During review, silently skipped modules were enabled by mapping their expected import names to numbered scripts. Tests containing outdated constructor arguments, text parameters in numerical checks, insufficient diversity samples or mistaken boundary conventions were repaired. The resulting suite checks actual routines rather than relying on the plausibility of their outputs.

The final audit corrected holding-period returns, price-path signal construction, overlapping positions, duplicate names, non-finite GARCH input, future-dependent percentile thresholds and unseeded GAN sampling. A simulated historical-validation placeholder was replaced with actual era-A backtests. An additional defect found during the rerun generated shocks before updating their conditional variance; it was corrected and pinned by a hand-calculated GARCH-recursion test. Earlier fidelity tests assumed a pooled-data schema and arbitrarily high kurtosis, so they were revised to validate the current single-asset output against statistics calculated independently from the frozen windows.

The final reported results were regenerated after these corrections. The test suite gives 79 passed and two expected failures. Those failures concern GAN output scale and non-deterministic training; keeping their checkpoints fixed makes the final sampling experiment reproducible but does not repair how the models were learned. A passing suite is evidence that the specified calculations satisfy their checks, not evidence that the generators are economically useful or that the statistical design has no limitations.

The earlier report and outputs are retained for provenance, with the pre-correction scripts archived separately. They are not combined with the corrected numerical tables. Real held-out scores were checked for equality across the four-generator evaluation, separate LSTM evaluation and historical baseline; all use the same 500 identifiers and frozen windows. This check ensures that differences between validation methods arise from their validation inputs rather than a silent change in the test outcome being ranked.

3.7 Reproducibility

The reproducibility repository is available at <https://github.com/georn/dsm500-synthetic-markets-real-decisions>. It contains the report notebook, code, frozen datasets, trained checkpoints and instructions for reproducing the reported results.

- **Environment.** Python 3.11.7. `requirements.txt` pins the direct dependencies (numpy 2.4.6, pandas 3.0.6, scipy 1.17.1, torch 2.14.0, yfinance 1.7.0 and others); `requirements-lock.txt` is a full `pip freeze` of the environment; `requirements-build.txt` adds the notebook and PDF tools. No credentials or environment variables are needed.
- **Data.** The frozen dataset is the pair of windowed-return files `windowed_data_era_a.parquet` and `windowed_data_era_b.parquet`, tracked in version control with their SHA-256 hashes recorded in the project README. Raw prices were not saved, the download date and yfinance version at download time were not recorded, and Yahoo data can be revised, so re-downloading may not reproduce these files exactly. Prices are adjusted closes (yfinance's default `auto_adjust=True`).
- **Randomness.** Seeds are fixed (`numpy.random.seed(42)` , `torch.manual_seed(42)`), which makes resampling and GARCH simulation repeatable. The final evaluation seeds GAN samples as well as classical simulations. GAN training is not deterministic (Section 3.6), so retraining gives different models; the trained weights used here are saved (`gan_model.pt` , `gan_model_lstm_fixed.pt`).
- **Running.** The README lists the scripts in order; Appendix E gives the code that reproduces the main table and the build commands. The RQ2 statistics and figures in this report come from `10_distinct_rule_rq2.py` and `11_report_figures.py`, which read saved outputs and run in under a minute. `15_refresh_report_tables.py` regenerates the marked tables from output files, avoiding manual transcription. `GENERATE_PDF.sh` executes this notebook and exports the PDF. A run manifest records data and checkpoint hashes for the corrected run.

3.8 Implementation

The study is implemented as a Python 3.11 pipeline of numbered scripts (Table 3.2). Each script reads from and writes to `outputs/`, so stages can be rerun independently. Numerical work uses NumPy, pandas and SciPy; the GANs use PyTorch; data are downloaded with yfinance and stored as Parquet files. All computation ran on a laptop CPU. The saved MLP GAN training took about two minutes and the LSTM about two hours; in the corrected rerun, evaluation took roughly one to two minutes per data source. These are observed local timings rather than portable performance guarantees.

Table 3.2. Pipeline stages, scripts and main outputs.

Stage	Script	Main output
Data collection and windowing	<code>01_data_pipeline.py</code>	<code>windowed_data_era_{a,b}.parquet</code>
Classical generators and RQ1 fidelity	<code>02_baselines.py</code>	<code>fidelity_results.csv</code>
Rule universe	<code>03_strategies.py</code>	rule list (in memory)
GAN training	<code>04_gan_generator.py</code> , <code>04b_gan_generator_lstm_FIXED.py</code>	<code>gan_model.pt</code> , <code>gan_model_lstm_fixed.pt</code>
Backtesting (RQ2)	<code>05_evaluation.py</code> , <code>05b_evaluation_lstm_comparison.py</code>	<code>evaluation_results*.csv</code>
Paired comparison of generators	<code>12_paired_rank_tests.py</code>	<code>paired_rank_tests.json</code>
GAN loss histories	<code>13_gan_training_history.py</code>	<code>gan_training_history.json</code>
RQ2 statistics and figures	<code>10_distinct_rule_rq2.py</code> , <code>11_report_figures.py</code>	<code>rq2_distinct_rules.json</code> , figures
Data description	<code>14_data_description.py</code>	<code>data_description.json</code> , Figure 3.2
Historical validation	<code>07_walkforward_baseline.py</code>	<code>walkforward_comparison.csv</code> , <code>walkforward_results.json</code>
Generated report tables	<code>15_refresh_report_tables.py</code>	updated marked tables in report notebook
Report build	<code>GENERATE_PDF.sh</code> , <code>tools/</code>	PDF and HTML

Data layout. Each stored window holds 256 daily log returns for one asset in a 768-column layout (three assets $\times$ 256 steps), with an asset label and a window identifier; the other assets' columns are empty. Helper functions in `05_evaluation.py` select single-asset windows (`asset_windows`) and rebuild the underlying series from overlapping windows (`reconstruct_series`).

Backtester. `StrategyEvaluator` applies one rule to one window. It computes the entry signal (a moving-average crossover, a momentum threshold or a percentile rule), applies the exit (a fixed holding period, or a profit target and stop loss within a maximum holding period)

and returns a Sharpe ratio. Values are then averaged over all real or synthetic windows. Appendix E.2 records the corrected return definition used for the reported results.

Rule generation. `generate_strategies` sweeps the parameter grids in a fixed order and stops at 500 specifications. Because moving-average rules come first, the Bollinger-band rules in the grid are never reached (Section 5.5).

Quality controls. The test suite (Section 3.6), fixed seeds, pinned dependencies (Section 3.7) and archived prior versions and a run manifest (Appendix D) support verification. Every module, class and function in the pipeline carries a docstring describing its inputs, outputs and conventions.

4. Results

4.1 RQ1: Stylized Facts Fidelity

The corrected fidelity comparison uses only the 788 era-A USD/EUR windows. It reports real windows, whole-window bootstrap, stationary bootstrap, GARCH and the two saved GANs. Values are means of per-window statistics, except W_1 , which measures distance between flattened window distributions. The GANs are sampled from their original pooled-data checkpoints; their inclusion measures those particular implementations rather than a clean comparison of architectures trained exclusively on the currency.

Table 4.1. Single-asset fidelity: mean daily standard deviation, mean autocorrelation over lags 1–20, excess kurtosis, skewness and first Wasserstein distance in log-return units.

Source	Daily σ	Return ACF	Squared-return ACF	Excess kurtosis	Skewness	W_1
Real era-A USD/EUR	0.00398	-0.012	0.027	2.48	0.32	0.000000
Window bootstrap	0.00399	-0.012	0.027	2.47	0.33	0.000008
Stationary bootstrap	0.00400	-0.008	0.018	2.05	0.30	0.000063
GARCH(1,1)	0.00396	-0.004	0.041	0.11	-0.00	0.000191
MLP GAN	0.00090	-0.003	-0.014	-0.44	-0.00	0.002286
LSTM GAN	0.00251	0.356	0.351	44.47	-4.23	0.001017

The window bootstrap closely preserves the observed moments because its paths are entire training windows. It does not create new within-window orderings. The stationary bootstrap retains the return scale but changes the composition of local sequences and slightly reduces tail thickness. Both are useful reference points, yet neither a low distributional distance nor preserved moments can establish a useful strategy-validation ranking.

GARCH's mean daily standard deviation is close to the real value, while its mean window excess kurtosis is only about 0.11 rather than 2.48. Its squared-return autocorrelation is positive under the repaired recursion, showing that conditional variance actually influences subsequent shocks. This is a calibrated-scale, normal-innovation model with fixed α and β ; it is not expected to reproduce every empirical tail or dependence property. The final model should therefore be described by the properties measured, rather than labelled generally realistic.

The MLP GAN's mean daily spread is about a quarter of the real window spread. The LSTM GAN has return autocorrelation around 0.36 and mean excess kurtosis around 44, compared with approximately -0.01 and 2.48 in real windows. Those distortions are direct evidence of

poor fidelity for these saved models. They remain compatible with the documented loss histories and the pooled, zero-filled training construction; they are not evidence that all deep generators would show the same defects.

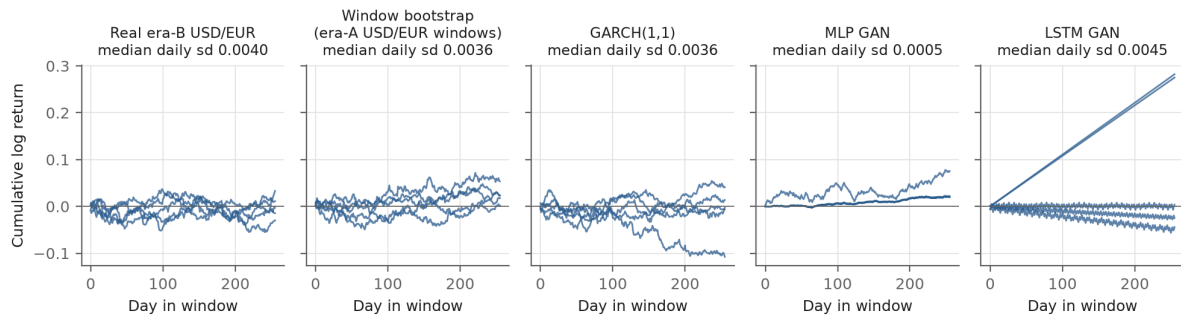


Figure 4.1. Five seeded sample paths per source, using USD/EUR columns. GARCH uses the corrected recursion and era-A variance targeting. Panel subtitles give the median daily spread of these five displayed paths.

The displayed paths complement the numerical table. They show MLP scale compression and nearly straight LSTM drift paths, while the fitted GARCH paths have a market-sized spread. Five paths cannot represent the full sampling distribution; the table uses hundreds of windows and is the quantitative fidelity result. Qualitative path realism is also a different question from whether a generator preserves differences between rule scores.

4.2 RQ2: Downstream Task-Based Utility (Primary Result)

All 500 uniquely named specifications are backtested with the corrected daily-position evaluator. Each method's validation score is compared with the same real era-B outcome. Marginal correlations use the rules with defined scores for that method: 488 for GARCH, bootstrap and historical validation, and 484 for the GANs. The chronological historical baseline uses actual era-A scores. Its file name retains the earlier walk-forward label, but its design is one fixed training/test split.

Table 4.2. Validation-to-real Spearman correlation, approximate 95% Fisher-z intervals and two-sided tests against zero. Values are generated from saved outputs.

Method	Spearman ρ	95% Fisher-z CI	p-value	n
GARCH(1,1)	0.041	[-0.048, 0.130]	0.362	488
Window bootstrap	-0.306	[-0.385, -0.224]	<0.001	488
LSTM GAN	0.031	[-0.058, 0.120]	0.495	484
MLP GAN	-0.020	[-0.109, 0.069]	0.658	484
Historical validation	-0.286	[-0.366, -0.202]	<0.001	488

GARCH, LSTM GAN and MLP GAN have correlations near zero; their intervals span zero and their tests do not resolve a positive or negative association. In this experiment they do not demonstrate useful agreement with later rankings. The distinction matters because the point estimates can still be ordered numerically: GARCH's slightly higher coefficient is not sufficient evidence that it is a superior validation method to the GANs.

Window bootstrap and historical validation have negative correlations. Rules that score well on the earlier histories tend to score less well in the later period. Window resampling creates additional samples of earlier windows but does not change their temporal environment, so its close agreement with the historical baseline is unsurprising. The observation is consistent with unstable strategy rankings across the chronological split; it does not identify the cause or establish that the earlier era was universally unsuitable for predicting any financial task.

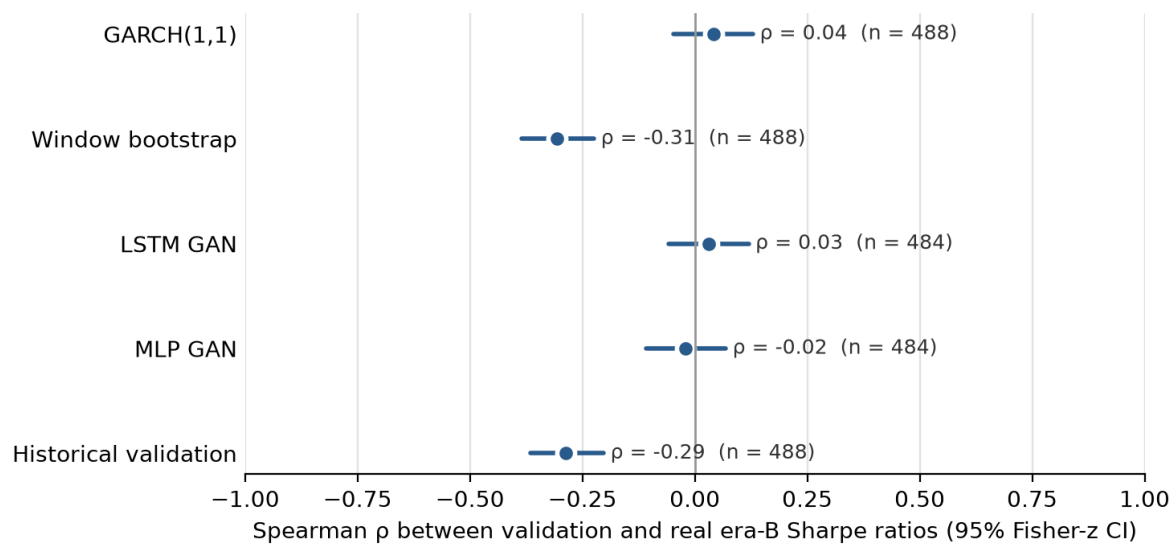


Figure 4.2. Spearman correlations and Fisher-z intervals for the four generators and the historical baseline. Negative correlations are displayed rather than clipped at zero.

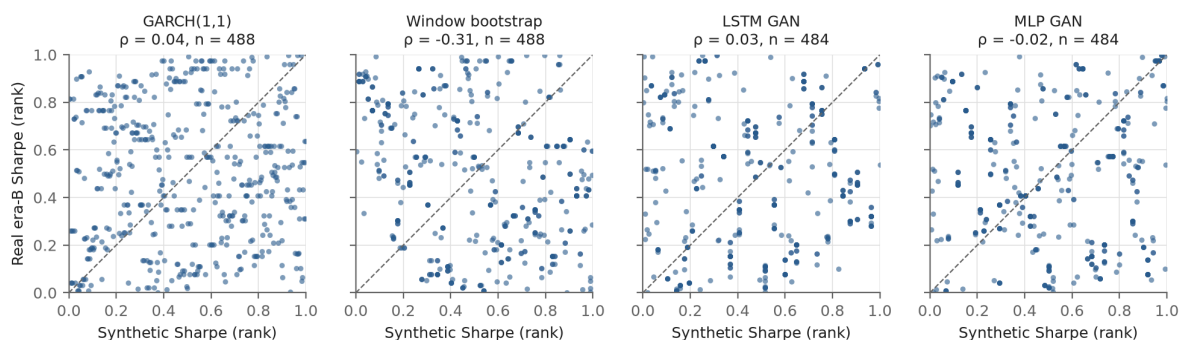


Figure 4.3. Real and synthetic percentile ranks for each generator. The diagonal represents perfect agreement; the bootstrap's reversed trend and the other generators' weak association explain the correlations.

The negative coefficients cannot be turned into a deployable strategy by reversing the rankings after observing era B. Such a decision would use the held-out results to select the direction of the method, creating a fresh form of overfitting. An inverse-ranking hypothesis would need an independently chosen rule and a further untouched period. The present result is evidence against the original expectation of straightforward positive ranking utility, not evidence of an exploitable trading opportunity.

The historical baseline improves the experiment even though its coefficient is poor. It makes the comparison concrete: GARCH and the GANs can be better than relying on the earlier ranking while still showing almost no absolute ranking information. Both aspects are reported. Paired comparisons in Section 4.4 test the differences on a common rule set so that missing momentum scores cannot by themselves explain the observed relative ordering.

4.3 RQ2 Secondary Metrics: False Discovery, Optimism and Selection Regret

The secondary metrics describe different decisions from the rank statistic. Endorsement uses a fixed score threshold of 0.1 in non-annualised daily-Sharpe units; optimism compares the magnitude of validation and real scores; regret examines the single rule selected by each method. They should not be collapsed into one overall verdict, because a method can have little ranking information yet happen to choose a comparatively good individual rule.

Table 4.3. Decisions under the fixed 0.1 daily-Sharpe screen, mean optimism and selection regret. A dash denotes an undefined FDR when no rules are endorsed.

Method	Endorsed	Above threshold on real data	FDR	Mean optimism	Selection regret
GARCH(1,1)	0	0	—	0.0226	0.0871
Window bootstrap	0	0	—	0.0317	0.1084
LSTM GAN	4	0	100%	-0.0008	0.0927
MLP GAN	26	0	100%	0.0770	0.0772
Historical validation	0	0	—	0.0315	0.1578

No rule reaches the threshold on real era-B data: the largest defined real score is about 0.080. GARCH, window bootstrap and historical validation endorse no rules, so their FDR is undefined rather than zero. The LSTM GAN endorses four rules and the MLP GAN twenty-six; every endorsement fails the same real-data threshold. Their 100% false-discovery fractions are therefore conditional on a stringent screen and a test universe with no qualifying real outcome. They do not independently establish that either model always makes worse decisions.

Mean optimism is positive for GARCH, bootstrap, historical validation and MLP GAN, while LSTM's mean bias is close to zero. Small mean bias does not imply accurate rankings: positive and negative errors can cancel. The MLP's optimism is largest, consistent with inflated strategy scores on paths whose return structure differs from the real data. These values are measured under the corrected daily-return definition and replace the extreme trade-based figures in earlier drafts.

The MLP-selected rule has the smallest descriptive regret in this particular test, despite the model's near-zero rank correlation. GARCH is not an oracle and does not minimise all decision losses. This inconsistency between metrics is informative: broad ranking accuracy, calibration, threshold screening and choosing one rule are different tasks. Their apparent agreement in the earlier defective analysis was not reproduced. No uncertainty analysis of selection regret was performed, so the ordering of these single realised regret values should not be treated as a general method ranking.

4.4 Robustness and Paired Comparison of Generators

A paired resampling comparison uses the 484 uniquely named specifications with defined real, synthetic and historical-validation scores. Each of 1,000 seeded resamples selects the same rule indices for every method, preserving the dependence between their correlation estimates. The common subset excludes momentum rules; it consists of 480 moving-average rules and four RSI-style variants. Its point correlations differ slightly from marginal correlations calculated on 488 rules.

Table 4.4. Correlations on the 484 common specifications with 95% percentile intervals from paired rule resampling.

Method	ρ on common rules	95% percentile interval
GARCH(1,1)	0.040	[-0.047, 0.121]
Window bootstrap	-0.308	[-0.384, -0.231]
LSTM GAN	0.031	[-0.064, 0.115]
MLP GAN	-0.020	[-0.124, 0.071]
Historical validation	-0.288	[-0.366, -0.209]

Table 4.5. Paired differences: mean difference, 95% percentile interval and fraction of resamples with a difference at or below zero. The final column is a resampling diagnostic, not a separately calibrated test p-value.

Difference in ρ	Mean	95% percentile interval	Resamples ≤ 0
GARCH(1,1) – Window bootstrap	0.346	[0.255, 0.432]	0.0%
GARCH(1,1) – LSTM GAN	0.009	[-0.083, 0.100]	42.6%
GARCH(1,1) – MLP GAN	0.058	[-0.050, 0.162]	13.8%
GARCH(1,1) – Historical validation	0.326	[0.232, 0.414]	0.0%
Window bootstrap – LSTM GAN	-0.337	[-0.459, -0.206]	100.0%
Window bootstrap – MLP GAN	-0.288	[-0.398, -0.175]	100.0%
Window bootstrap – Historical validation	-0.020	[-0.031, -0.010]	100.0%
LSTM GAN – MLP GAN	0.049	[-0.089, 0.187]	25.3%
LSTM GAN – Historical validation	0.317	[0.186, 0.437]	0.0%
MLP GAN – Historical validation	0.268	[0.148, 0.377]	0.0%

The GARCH-minus-bootstrap interval is above zero, as are the GARCH-minus-historical and corresponding GAN-minus-historical intervals. However, intervals for GARCH against either GAN, and for one GAN against the other, include zero. The defensible relative finding is a separation between negatively ranked historical methods and methods with approximately null rankings, rather than a unique GARCH victory.

Window bootstrap is slightly more negative than historical validation on this common subset. The small difference is narrowly resolved by this resampling procedure because the two methods' rule scores are closely related. It does not establish a material economic cost, and the independence assumption for rule indices makes the nominal interval optimistic. A different parameter universe, market period or resampling unit could change this fine ordering.

The test does not resample whole market histories, refit generators or quantify uncertainty from GAN training. It therefore leaves the central single-history limitation intact. A family-level resampling design or repeated untouched test periods would give a stronger basis for claims about methods. The present intervals are reported as conditional evidence from one chosen universe and are accompanied by that qualification wherever a pairwise comparison is interpreted.

4.5 Sensitivity by Rule Family

Table 4.6. Correlations by rule family. Counts show specifications with a defined real score against the number generated; a dash denotes fewer than ten complete observations for a

method, for which no family correlation is reported.

Rule family	Real defined / specified	GARCH(1,1)	Window bootstrap	LSTM GAN	MLP GAN	Historical validation
MA crossover	480/480	0.032	-0.308	0.021	-0.033	-0.286
Momentum	4/16	—	—	—	—	—
RSI-style	4/4	—	—	—	—	—

The MA-only correlations preserve the broad full-set result: window bootstrap and historical validation are negatively ranked, and the other generators have little detectable ranking signal. This confirms that four rare momentum rules or four RSI variants are not driving the main pattern. It does not demonstrate robustness across genuinely different strategy classes, since MA rules dominate both analyses.

Only four momentum specifications have defined real scores, and no momentum specification is available for the saved GAN comparisons. The four RSI-style rules are close exit variants rather than a broad exploration of mean-reversion designs. Numerical family correlations from those tiny groups would be fragile and potentially misleading, so they are omitted. A balanced strategy universe would be needed to investigate whether the observed failure is specific to trend-following logic.

4.6 Summary of Results

Table 4.7. Research-question outcomes after the corrected rerun.

RQ	Evaluation	Finding
RQ1	Single-asset fidelity	Window bootstrap closely matches training moments; GARCH matches scale but not tails; both saved GANs have material fidelity defects.
RQ2	Ranking and decision utility	No detectable positive ranking signal from GARCH or either GAN; bootstrap and historical validation are negatively correlated with later rankings. Relative improvement over those baselines does not establish a useful generator.
RQ3	Assets and regimes	Deferred; the descriptive era contrast is not a regime-stratified utility experiment.

The three layers of analysis answer different questions. Fidelity describes the generated distribution, rank correlation describes ordering across rules, and the decision metrics describe a fixed threshold or a single selected rule. The experiment's contribution is keeping those questions separate. In particular, the near-null generators should not be promoted as useful simply because they beat negatively correlated comparators, and the closely matched bootstrap should not be promoted because its resemblance measures look strong.

5. Discussion

5.1 Why Does Matching History Not Preserve Later Rankings?

The window bootstrap has strong fidelity on the measured training moments and a negative downstream correlation. Its construction explains why those observations can coexist. Every path is a complete era-A window, so resampling adds repetition and changes the weighting of earlier episodes without introducing a new local return order. A rule that exploits a feature of the earlier history will tend to retain that feature in resampled validation data, even when the rule performs differently in the later test period.

Historical validation produces a similar negative correlation, supporting that interpretation at the level of rankings. It suggests that replaying earlier performance did not give a stable ordering for this mostly moving-average universe across the selected era split. Table 3.1 shows that the currency's volatility and squared-return dependence differ between eras, but the experiment does not isolate whether those changes, return drift, trend structure, exit sensitivity or another property produced the reversal. A plausible explanation is not a measured causal mechanism.

GARCH generates new paths rather than replaying earlier ones and retains clustered variance under its corrected recursion. Its near-zero correlation could reflect removal of misleading era-specific ordering, failure to model the conditional return structure that the rules need, or both. The data do not distinguish those explanations. A method that removes an adverse association can improve on a comparator without learning the positive relationship required for reliable ranking. That is why the comparison must retain the zero-correlation benchmark as well as the historical baseline.

There is a related distinction between conditional volatility and expected return. The GARCH model specifies how variance depends on past shocks but draws innovations with zero conditional mean around a fixed drift. Moving-average rules depend on path order and trends; reproducing return scale and volatility persistence does not guarantee that those features are correctly represented. Its small tail thickness in Table 4.1 further demonstrates that variance targeting addresses one property rather than a complete financial-data model. The negative result is therefore compatible with a correctly implemented yet task-incomplete generator.

A stronger test of this interpretation would vary one feature at a time in a known data-generating process, fit the same generators and compare rankings against a known optimum. That synthetic ground-truth control was planned but is not delivered. The current study can show disagreement between its validation methods and one realised future; it cannot prove whether a generator fails to preserve a true strategy advantage or the held-out history is itself an unusual draw. The distinction limits causal and population-level conclusions while leaving the empirical failure of the chosen validation task clear.

5.2 Why the Deep Generators Do Not Demonstrate Utility

Neither GAN shows a resolved positive correlation with real rankings. Their mutual difference and differences from GARCH are unresolved in the paired comparison. The LSTM's small positive point estimate therefore does not demonstrate an advantage from temporal architecture, and the MLP's slightly negative one does not prove a systematic inverse ranking. Interpretation must begin with this uncertainty rather than an ordered list of point estimates.

Several implementation limitations prevent a broad verdict on deep generation. Both checkpoints were trained on a pooled 768-column representation with two thirds of each row empty and zero-filled. The effective diversity of the currency input is also much smaller than its 788-window count, because adjacent windows share 255 observations. There was no hyperparameter search, repeated-seed training comparison or architecture selection based on a held-out development period. The generator's adversarial objective did not directly optimise rule-ranking preservation.

The fidelity diagnostics supply evidence beyond the downstream scores. The MLP generates returns with compressed scale, and the LSTM exhibits high serial dependence and very heavy window tails that are absent in the observed currency windows. The saved loss histories likewise do not support a claim of well-balanced training: the MLP discriminator dominates, while the LSTM losses stay near chance values. A stronger training protocol might change both resemblance and decision quality. This study establishes the performance of these particular checkpoints under a corrected evaluator rather than a limitation intrinsic to GANs.

Their relative performance against historical validation needs equally careful wording. Both GANs score above the negatively correlated historical baseline in paired rule resampling, but their own rank correlations are essentially zero. A policy that selected a generator solely because it won that relative comparison could therefore still obtain no reliable positive ordering. This is a practical reason to specify an absolute usefulness criterion before comparing tooling costs or discussing deployment.

The secondary metrics show why a single score cannot rescue the conclusion. LSTM has little mean optimism, yet no useful ranking signal; MLP has the lowest single-selection regret but endorses many rules that fail the chosen threshold. Calibration in the mean, selecting one comparatively good rule and ranking a broad set are distinct achievements. The experiment does not support a universal model ordering across them, and their divergence argues for evaluating the downstream decision actually intended by a user.

5.3 Relation to Prior Work

Fidelity and usefulness. Assefa et al. (2019) caution that resemblance measures cannot by themselves establish realism. The corrected experiment adds a direct downstream observation: resampled paths closely match the measured training moments while their rule

rankings are negatively associated with later performance. This is a narrower claim than saying statistical realism never helps. The study measures a small set of moments and one ranking task; useful structure might exist in unmeasured properties or a different decision setting.

Classical and deep generators. Quant GANs (Wiese et al., 2020) and the financial-return GAN of Takahashi, Chen and Tanaka-Ishii (2019) demonstrate that deep generators can model stylized facts. The present MLP and LSTM implementations are different architectures and have documented training problems, so their weak results do not refute those studies. Likewise, a variance-targeted GARCH with fixed persistence is not a fully estimated volatility model. The fair conclusion is that neither implementation supplies resolved positive utility here, rather than that cheap models universally outperform learned models.

Task-based evaluation. Koshiyama, Firoozye and Treleaven (2021) study conditional GANs for strategy fine-tuning and combination across a much larger asset setting. Their task, generator and scale differ, so their reported benefits can coexist with this negative result. Together, the studies motivate declaring the target decision before choosing evaluation criteria. A generator helpful for calibration, forecasting or augmentation need not preserve the ranking of fixed long-only rules across two market eras.

Overfitting corrections. White (2000), Bailey and López de Prado (2014), and Bailey et al. (2017) address selection and testing against a realised historical record. Synthetic histories do not remove that problem automatically. Here, final usefulness is still judged against one era-B history, and choosing a preferred model, parameter set or rank direction after seeing its outcome would reuse that test information. The experiment therefore evaluates a specified framework on one split rather than establishing an out-of-sample deployment policy.

A useful negative result. The proposal anticipated an informative negative outcome. The corrected experiment provides one: neither deep generator nor GARCH demonstrates positive ranking utility, despite some relative improvements over historical methods. This outcome constrains claims about tooling value and motivates better model training and stronger controls. It also shows why honest reporting of a correction is substantive research work: preserving an attractive conclusion after discovering its faulty basis would misstate the evidence.

5.4 Limitations

Saved GAN training. The checkpoints were not retrained on clean single-asset inputs. Their pooled, zero-filled representation, limited effective sample diversity and unsuccessful training dynamics remain central constraints. The final seeded sampling and corrected evaluator make their measurements auditable but do not constitute a fair architecture-level comparison with well-tuned deep financial generators.

One asset and future history. The task concerns USD/EUR, a mainly MA universe and one chronological split. Descriptive differences in volatility between eras do not substitute for the deferred regime-stratified RQ3 experiment. SPY and the FTSE 100 are prepared but not evaluated. Performance in a different pair, quote direction or held-out period remains unknown.

Overlapping windows and related rules. Windows share nearly all observations, and many rule variants differ only in an exit parameter. The nominal counts are not independent sample sizes. Fisher intervals and rule-resampling intervals therefore understate sources of uncertainty relevant to other periods or strategy populations. Resampling market blocks, entry/exit families or repeated independent periods would address different parts of this problem.

Conditional score averaging. No-trade and zero-variation windows are excluded from mean Sharpe calculations, so average scores can depend on a rule's activity. Only 488 specifications have a defined real score, and the all-method subset has 484. This missingness is reported but not itself modelled. An experiment based on continuous daily portfolios or an explicit no-trade scoring convention could yield different rankings.

Simplified economics and model fitting. Trading is long-only with daily closing signals, zero costs and no risk-free adjustment. The evaluator uses log-return thresholds and cumulative log-price moving averages. GARCH targets the observed variance while retaining fixed α and β and Gaussian innovations; it does not estimate all parameters by likelihood or model changing conditional means. The results concern these choices rather than production execution under spreads, slippage or leverage.

Decision and control limits. The endorsement threshold is fixed at a daily Sharpe of 0.1 rather than derived from a statistical test. No real rule exceeds it, making GAN false-discovery fractions conditional on a universe with no qualifying outcome. Selection regret is a descriptive one-period value with no confidence interval. The promised synthetic ground-truth control is not implemented, so the true best rule under a known process is not available.

Inference after correction. The final run repairs the known numerical defects, but it follows an audit that observed the earlier outputs. The rerun does not create an independent new test period. Consequently, corrected tables should not be interpreted as an additional replication or used to tune an inverse-ranking policy without further data. The earlier apparent positive ranking evidence is superseded, not pooled with the current measurements.

These limitations preserve the value of the negative result while restricting its reach. The contribution is a transparent evaluation on a specified task with reproducible calculations and documented implementation history. It does not establish that synthetic financial data is generally useless, that one architecture is intrinsically superior, or that reversed historical rankings offer a profitable trading opportunity.

5.5 Changes from the Proposal

Table 5.1 sets out where the delivered study departs from the proposal.

Table 5.1. Planned and delivered design.

Element	Proposal	This study
Data	Hourly BTC/USDT and ETH/USDT as primary data; daily EUR/USD, SPY and a FTSE 100 fund for a cross-asset arm	Daily USD/EUR only (Yahoo $\text{EUR}=\text{X}$, quoted as euros per dollar); SPY and the FTSE 100 index, not the tradable fund, prepared but not evaluated
Deep generator	Temporal-convolutional GAN in the Quant GAN style	MLP GAN and LSTM GAN
Resampling baseline	Stationary (block) bootstrap	Whole-window bootstrap
Rule families	Moving-average crossover, RSI, Bollinger bands	Moving-average crossover, momentum, RSI-style; Bollinger rules not reached
False-discovery threshold	Sharpe ratio significantly above zero at the 5% level	Fixed Sharpe threshold of 0.1
Synthetic ground-truth control	Known data-generating processes with a known best rule	Not implemented
Transaction costs	Exchange fees and spreads	Not modelled
Comparing methods	Tests for differences in predictive accuracy	Paired resampling of differences in ρ , including the historical baseline (Section 4.4)
Walk-forward baseline	Single-history walk-forward validation	Actual era-A versus era-B backtests; one chronological split, not repeated rolling refits
RQ3	Extension	Deferred

The data change followed a failed attempt to collect the hourly cryptocurrency data through the ccxt library; daily data for the cross-asset instruments were already available. The proposal also expected GARCH to be the weaker comparator, because with near-independent innovations it cannot reproduce conditional return predictability. The corrected outcome supports no detectable positive ranking signal from GARCH or either GAN. GARCH performs better than the negatively ranked bootstrap but not demonstrably better than the GANs. This replaces an apparent earlier advantage produced by defective calculations and is a reminder to test assumptions about both method strength and implementation correctness.

Response to proposal feedback. The feedback asked for:

- a conceptual workflow diagram (now Figure 3.1);
- critical synthesis across studies, including recent diffusion-based work (Section 2);
- justification of the window length (Section 3.2) and of the assets (Section 3.1).

The feedback allowed the cross-asset analysis to remain optional; it is deferred with RQ3.

5.6 Reflection

What worked. The task-based framing turned a vague question (is synthetic data useful?) into a measurable one, and the classical baselines gave the comparison a meaningful reference point. The chronological split and the saved, hashed datasets made the analysis auditable, which is what allowed its defects to be traced.

What went wrong. Several defects in the initial pipeline were found only during the final review and corrected before regenerating the final results:

- trade returns were computed incorrectly;
- rule specifications that shared a name were merged;
- the walk-forward baseline was a simulation;
- a GARCH parameter file did not come from any script;
- the GANs were trained on zero-filled multi-asset rows;
- the GARCH recursion generated each shock before updating its conditional variance.

Three test modules that would have exposed some of these had been silently skipped because of import errors, so a passing test run gave false assurance. Most of the defects share one cause: outputs were checked for plausibility (finite values, sensible ranges) rather than against independent calculations or invariants.

What would be done differently.

- Run the full test suite automatically from the start, treating skipped tests as failures.
- Assert invariants such as unique identifiers where the data are created.
- Test numerical routines against hand-computed examples, as `tests/test_known_defects.py` now does.
- Keep placeholder results out of the main pipeline.
- Record the code version that produced every output file.
- Plot generated paths early; Figure 4.1 would have revealed the GANs' scale problem before any downstream analysis.

Scope. The project attempted more than the time allowed: three assets, four generators and three research questions. A narrower design, with one asset, fewer generators and a corrected backtester checked twice, would have produced firmer conclusions within the same time.

6. Conclusions

6.1 Answering the Research Questions

RQ1: statistical fidelity. The corrected comparison evaluates all four generators on single-asset windows, with stationary bootstrap as an additional fidelity reference. Window resampling closely preserves measured training moments; GARCH preserves the return scale and clustered variance but under-represents tails. The saved MLP and LSTM GANs distort scale or serial dependence. These measurements replace pooled and incomplete statistics in the earlier pilot.

RQ2: downstream usefulness. GARCH, LSTM GAN and MLP GAN have correlations close to zero, with intervals spanning zero. Window bootstrap and actual historical validation are negatively associated with later scores. GARCH and the GANs therefore perform better than those historical comparators in paired resampling, but none demonstrates a reliable positive ranking. GARCH cannot be separated from either GAN on the common subset. Decision metrics also disagree: low mean bias or lower single-selection regret does not supply broad ranking accuracy.

RQ3: assets and regimes. Deferred. Differences between era-A and era-B descriptive volatility provide context but do not answer whether validation utility changes across regime strata or assets.

6.2 Contribution

The methodological contribution is an auditable task-based framework that separates financial-data resemblance, absolute ranking agreement, relative method comparisons and specific selection decisions. It retains an actual historical benchmark and allows generators to fail the downstream test even when some fidelity diagnostics look strong.

The empirical contribution is a negative result on a defined case study. A closely moment-matched window bootstrap reproduces an earlier ranking that reverses in the held-out period, while a variance-targeted GARCH and two saved GANs show little positive ranking information. This provides a reason to require direct decision evidence before adopting synthetic validation rather than treating realistic paths as sufficient.

The corrective process is also documented. The final calculations use proper daily-position returns, unique identifiers, a corrected GARCH recursion and computed historical-validation scores. Their changed findings show that plausible numerical outputs and an initially incomplete test suite were insufficient assurance. The earlier apparent GARCH advantage has been replaced with the evidence produced by the audited pipeline.

6.3 Practical Implications

A practitioner should evaluate a generator on the intended downstream decision and report both its absolute signal and its comparison with existing methods. A win over a negatively correlated comparator does not establish a useful screening tool. Neither the corrected GARCH nor the saved GANs should be recommended for selecting the tested rules on the strength of this experiment.

Synthetic validation should be treated as supplementary evidence alongside untouched real-data evaluation, clearly declared transaction assumptions and controls against repeated selection. Statistical fidelity remains relevant, but it is insufficient on its own. Reporting coverage, undefined decisions and model-training quality makes a negative result useful to researchers considering where further effort is justified.

6.4 Future Work

Priority should be given to retraining the GANs on clean currency windows, logging convergence and evaluating repeated seeds without selecting them on era-B results. Fully estimating GARCH, adding conditional-mean or regime dynamics, and comparing a stationary bootstrap on the downstream task would investigate whether the failure arises from model choice or the ranking problem itself.

A synthetic ground-truth control should establish the ranking under known return processes before generators are fitted to finite samples. This can distinguish a failure to learn decision-critical structure from noise in one realised held-out market. Transaction costs, a balanced rule universe, rolling chronological validation and additional assets or untouched periods are needed before considering any practical deployment claim. Those extensions remain future work; the present study's conclusions are limited to the corrected experiment delivered here.

7. Ethical Considerations

Data and participants. The study uses only publicly available market prices for USD/EUR, SPY and the FTSE 100, downloaded from Yahoo Finance with the open-source yfinance library (Yahoo Finance, 2026; Aroussi, n.d.). No human participants were involved, and no personal or confidential data were collected, so the work falls outside human-participant research ethics.

Terms of use and redistribution. yfinance is an independent tool, neither affiliated with nor endorsed by Yahoo. The data were used for non-commercial academic research. Raw price files were not retained; only derived windowed returns are stored in the project repository.

Credentials and security. Access to an IG demo account was tested at the proposal stage, with credentials kept in an untracked environment file excluded from version control. No IG data were used for the reported results, and no credentials appear in the code, outputs or report.

Responsible use and reporting. The project makes no investment recommendation and trades no capital. Synthetic data carries a mild dual-use risk: it could be used to overstate a strategy's robustness. The project addresses that risk by measuring when synthetic validation does and does not predict real performance, and by reporting its own limitations openly, including the pipeline defects found during the final review (Sections 3.6 and 5.4).

Reproducibility and research integrity. Code, tests, pinned dependencies and a hashed copy of the dataset are provided (Sections 3.6–3.7). The scripts are also archived exactly as they produced the reported results (Appendix D), so the findings can be checked independently.

8. References

- Aroussi, R. (n.d.) *yfinance: download market data from Yahoo! Finance's API* [Python library], version 1.7.0. Available at: <https://github.com/ranaroussi/yfinance> (Accessed: 5 October 2026).
- Assefa, S., Dervovic, D., Mahfouz, M., Balch, T., Reddy, P. and Veloso, M. (2019) 'Generating synthetic data in finance: opportunities, challenges and pitfalls', *NeurIPS 2019 Workshop on AI in Financial Services*. Available at SSRN: <https://ssrn.com/abstract=3634235>. (A revised version appeared in the *Proceedings of the First ACM International Conference on AI in Finance*, 2020, doi:10.1145/3383455.3422554, with R.E. Tillman in place of T. Balch.)
- Bailey, D.H., Borwein, J., López de Prado, M. and Zhu, Q.J. (2017) 'The probability of backtest overfitting', *Journal of Computational Finance*, 20(4), pp. 39–69.
- Bailey, D.H. and López de Prado, M. (2014) 'The deflated Sharpe ratio: correcting for selection bias, backtest overfitting, and non-normality', *Journal of Portfolio Management*, 40(5), pp. 94–107.
- Bollerslev, T. (1986) 'Generalized autoregressive conditional heteroskedasticity', *Journal of Econometrics*, 31(3), pp. 307–327.
- Cont, R. (2001) 'Empirical properties of asset returns: stylized facts and statistical issues', *Quantitative Finance*, 1(2), pp. 223–236.
- Fama, E.F. (1970) 'Efficient capital markets: a review of theory and empirical work', *The Journal of Finance*, 25(2), pp. 383–417.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative adversarial nets', *Advances in Neural Information Processing Systems*, 27, pp. 2672–2680.
- Harvey, C.R., Liu, Y. and Zhu, H. (2016) '... and the cross-section of expected returns', *The Review of Financial Studies*, 29(1), pp. 5–68.
- Ho, J., Jain, A. and Abbeel, P. (2020) 'Denoising diffusion probabilistic models', *Advances in Neural Information Processing Systems*, 33, pp. 6840–6851.
- Huang, H., Chen, M. and Qiao, X. (2024) 'Generative learning for financial time series with irregular and scale-invariant patterns', in *International Conference on Learning Representations (ICLR 2024)*.
- Koshiyama, A., Firoozye, N. and Treleaven, P. (2021) 'Generative adversarial networks for financial trading strategies fine-tuning and combination', *Quantitative Finance*, 21(5), pp. 797–813.
- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.

Politis, D.N. and Romano, J.P. (1994) 'The stationary bootstrap', *Journal of the American Statistical Association*, 89(428), pp. 1303–1313.

Takahashi, S., Chen, Y. and Tanaka-Ishii, K. (2019) 'Modeling financial time-series with generative adversarial networks', *Physica A: Statistical Mechanics and its Applications*, 527, 121261.

White, H. (2000) 'A reality check for data snooping', *Econometrica*, 68(5), pp. 1097–1126.

Wiese, M., Knobloch, R., Korn, R. and Kretschmer, P. (2020) 'Quant GANs: deep generation of financial time series', *Quantitative Finance*, 20(9), pp. 1419–1440.

Yahoo Finance (2026) *Historical daily prices for EUR=X, SPY and ^FTSE, 2018–2024*. Available at: <https://finance.yahoo.com> (Accessed: September 2026).

Yoon, J., Jarrett, D. and van der Schaar, M. (2019) 'Time-series generative adversarial networks', *Advances in Neural Information Processing Systems*, 32.

Yuan, X. and Qiao, Y. (2024) 'Diffusion-TS: interpretable diffusion for general time series generation', in *International Conference on Learning Representations (ICLR 2024)*.

9. Appendices

A. Data Summary

Table A.1. Windows (256 daily log returns, stride 1) and underlying return counts by asset and era.

Asset	Era-A windows	Era-B windows	Era-A returns	Era-B returns
USD/EUR	788	526	1,043	781
SPY	752	497	1,007	752
FTSE 100	756	499	1,011	754

Windows are 256 daily log returns with stride 1 (`outputs/data_metadata.json`). Return counts are the underlying sequence lengths reconstructed from the windows; they are not independent observations.

B. GAN Training

Per-epoch losses are read from the saved checkpoints by

`13_gan_training_history.py` (`outputs/gan_training_history.json`).

MLP GAN: 50 epochs, about 2 minutes, trained on all 2,296 pooled era-A rows. The discriminator loss fell from 1.21 to 0.10 while the generator loss rose from 0.68 to 6.42, which means the discriminator overpowered the generator rather than the two reaching a balance. An earlier summary file (`gan_training_summary.json`) gave different starting values (0.821 and 5.29). No script produced that file, and its reference statistics match no dataset, so it is not used.

LSTM GAN: 50 epochs, about 120 minutes. The discriminator loss stayed at 1.386–1.385 and the generator loss at 0.686–0.696 throughout, close to $2 \ln 2$ and $\ln 2$, the values for a discriminator guessing at chance; the losses show no learning. The corrected daily-position evaluator no longer produces the earlier extreme trade-based optimism figures. Nevertheless, its fidelity diagnostics show excessive serial dependence and heavy tails, and its rank correlation is near zero. That distinction separates a repaired scoring routine from unrepaired generator training.

C. Rule Universe

500 uniquely named specifications: 480 MA crossover, 16 momentum and 4 RSI-style. Defined real scores: 488 (480 MA, 4 momentum, 4 RSI-style). The all-method comparison uses 484 specifications (480 MA and 4 RSI-style).

D. Code Availability

Appendix E lists the code that reproduces the main results table and builds the report.

- `01_data_pipeline.py` : data collection, cleaning, windowing
- `02_baselines.py` : stationary bootstrap and GARCH generators; RQ1 fidelity
- `03_strategies.py` : rule generation
- `04_gan_generator.py` , `04b_gan_generator_lstm_FIXED.py` : MLP and LSTM GAN training
- `05_evaluation.py` , `05b_evaluation_lstm_comparison.py` : per-rule real and synthetic Sharpe ratios
- `06_bootstrap_resampling_validation.py` : earlier three-generator resampling check (superseded by `12_paired_rank_tests.py`)
- `12_paired_rank_tests.py` : paired resampling comparison of the generators and historical validation (Tables 4.4 and 4.5)
- `13_gan_training_history.py` : per-epoch GAN losses from the saved checkpoints (Appendix B)
- `14_data_description.py` : descriptive statistics and Figure 3.2 (Section 3.1)
- `10_distinct_rule_rq2.py` : RQ2 statistics on distinct rules (Section 4.2) and Figure 4.2
- `11_report_figures.py` : Figures 3.1, 4.1 and 4.3
- `07_walkforward_baseline.py` : computed chronological historical-validation baseline (single train/test split)
- `tests/` : pytest suite (Section 3.6); `README.md` : environment, data hashes and run order
- `archive/pipeline_as_run_2026-10-05/` : earlier defective scripts, retained for provenance and not used for final reported numbers
- `15_refresh_report_tables.py` : refreshes marked report tables from corrected output files
- `outputs/corrected_run_manifest.json` : frozen-data and checkpoint hashes, corrected source hashes and run configuration
- `archive/pre_corrected_rerun_2026-10-05/` : report and numerical outputs before the corrected rerun

E. Code Listings

E.1 Reproducing Table 4.2. The cell below reads the RQ2 statistics produced by `10_distinct_rule_rq2.py`. The main report tables are refreshed from output files by `15_refresh_report_tables.py`; they are not manually transcribed.

```
import json
import pandas as pd

r = json.load(open('outputs/rq2_distinct_rules.json'))
rows = {}
for name, g in r['generators'].items():
    lo, hi = g['fisher_ci']
    rows[name] = {'n': g['n'], 'rho': round(g['spearman_rho'], 3),
                  'CI low': round(lo, 3), 'CI high': round(hi, 3)}
pd.DataFrame(rows).T
```

	n	rho	CI low	CI high
garch	488.0	0.041	-0.048	0.130
bootstrap	488.0	-0.306	-0.385	-0.224
lstm	484.0	0.031	-0.058	0.120
gan	484.0	-0.020	-0.109	0.069
wf	488.0	-0.286	-0.366	-0.202

E.2 Daily-position return and Sharpe definition used for the reported results. A signal at the close of the entry day creates exposure from the next day. At most one position is held; the full holding-period daily return stream enters the non-annualised window score. Profit and stop checks use cumulative held log returns. Undefined windows are excluded from the rule-level mean.

```
position[entry_idx + 1:exit_idx + 1] = 1.0
path_return = returns[entry_idx + 1:i + 1].sum()
daily = position * returns
sharpe = daily.mean() / daily.std() # if a position and nonzero
variation exist
```

E.3 GARCH recursion. For each path, the initial variance equals the fitted daily variance and the first shock is stochastic. The variance is updated before generating each later innovation:

```
eps[0] = np.sqrt(h[0]) * z[0]
for t in range(1, n_steps):
    h[t] = omega + alpha * eps[t - 1]**2 + beta * h[t - 1]
    eps[t] = np.sqrt(h[t]) * z[t]
returns = mu + eps
```

The regression suite checks the calculation against independently constructed innovations and variances. The archived earlier recursion is retained as provenance, and the final output file records the actual variance-targeted parameters.

E.4 Build. Regenerate output tables, execute the notebook and export the report with the pinned project environment:

```
.venv/bin/python pipeline/15_refresh_report_tables.py  
./GENERATE_PDF.sh
```